Lab 4 - Flappy Bird

Visible chat transcript and implementation record

Prepared 12 October 2026. This export contains the task conversation visible to the user through the packaging update. Tool commands, internal instructions and later messages are excluded; the appendix separately records implementation and verification. No additional student prompts or student code reviews have been invented.

User - attached materials

Lab_4_VibeCoding_Student_handout.pdf was attached. Assigned repository: https://github.com/SETAPESU26/40_flappy-bird

User - request

I've uploaded the files & the github's repo link, I want ya to finish everything for me

Assistant

I'll read the lab handout and inspect the repository, then complete the required code and submission materials and verify what runs.

Assistant

The repo needs four changes: accurate collision detection, a game-over screen, replay with three difficulty levels, and sound effects. The handout also requires before/after videos and a chat-history document.

I'm using the PDF skill to check the handout and prepare the chat export. I'll capture the original code before editing, then implement and test each feature. Uploading to GitHub will require your personal lab-repo URL; the handout explicitly forbids a PR to the class repo.

Visible transcript - continued

Assistant

The original collision check only tests the bird's centre, so its visible edge can overlap a pipe without registering a hit. I've replaced that with a circle-to-pipe check and small movement steps to prevent fast pipes from skipping collisions.

The game now has difficulty selection, a final-score screen, replay, exit controls, and generated sound effects. Next I'm checking those behaviours and recording the after video. Both videos use disclosed scripted inputs in this cloud environment.

Assistant

All 18 tests pass, including edge collisions, fast-pipe collisions, scoring, replay resets, and missing-audio-device handling. Both videos are exactly 10 seconds; the after video shows scoring, the repaired edge collision, and replay at all three difficulties, with sound.

I'm packaging the code, videos, chat transcript, and Mac setup instructions into a Lab-4 folder. The GitHub upload is still pending because I only have the class-repo URL.

End of visible transcript

The following page is an assistant-prepared technical record, not additional conversation.

Implementation and verification appendix

Source: SETAPESU26/40_flappy-bird, commit a6057332daff3c0f10deb4367eb830a12445bd0a. Original source is retained under tools/original; original requirements are retained in ORIGINAL_README.md.

1. Collision detection

Replaced centre-only detection with closest-point circle/rectangle geometry. Contact is a collision. Float coordinates avoid truncation. Movement is subdivided into steps of at most one pixel per axis to prevent fast pipes passing through undetected.

2. Game over

Added menu, playing and game_over states. A ground, ceiling or pipe hit freezes the round, displays the final score and reason, and waits for input. A death effect plays once.

3. Replay and difficulty

Keys 1/2/3 or buttons start Easy/Medium/Hard, with speed/gap settings of 3/200, 4/165 and 6/135. Every replay clears the score, velocity, pipes, timer and reason. Esc/Q and a clickable exit control close play.

4. Sound feedback

Added locally synthesized flap, score and death effects. No external assets are needed. M toggles sound. Unavailable audio hardware is handled without crashing.

Verification

18 automated tests passed on Linux / Python 3.12.14 / Pygame 2.6.1 using SDL dummy video/audio. Tests include edge and corner geometry, fast motion, every death cause, one-time scoring, replay resets, input, spawning, empty pipes, rendering and audio fallback. Entry-point startup/QUIT cleanup, compilation and whitespace checks also passed.

Iteration record

The first after-video setup collided before scoring; its controlled gap placement was corrected. The recorder also exposed an empty-pipe-list error in the movement-step calculation. That error was fixed and a regression test added. The final recording contains flap, score and death events.

Video evidence

Both clips are 600 frames at 60 fps (10 seconds). They show actual engine rendering under scripted input with disclosed controlled pipe placements. After-video audio is mixed from the same generated samples at recorded event frames. These are automated recordings, not recordings of a student playing.

Remaining actions and limits

Mac hardware display/audio has not been tested here. Publishing requires the personal lab repository and authenticated GitHub access. No changes were pushed and no PR was raised to the class repo. The source README also requests a shared conversation URL; obtain it using ChatGPT Share if needed.